

Report

Task 1: Automated bidding model for used car auction



NAME: TAKU TAJO JULY
ROLL NO.: 250104095

Summary

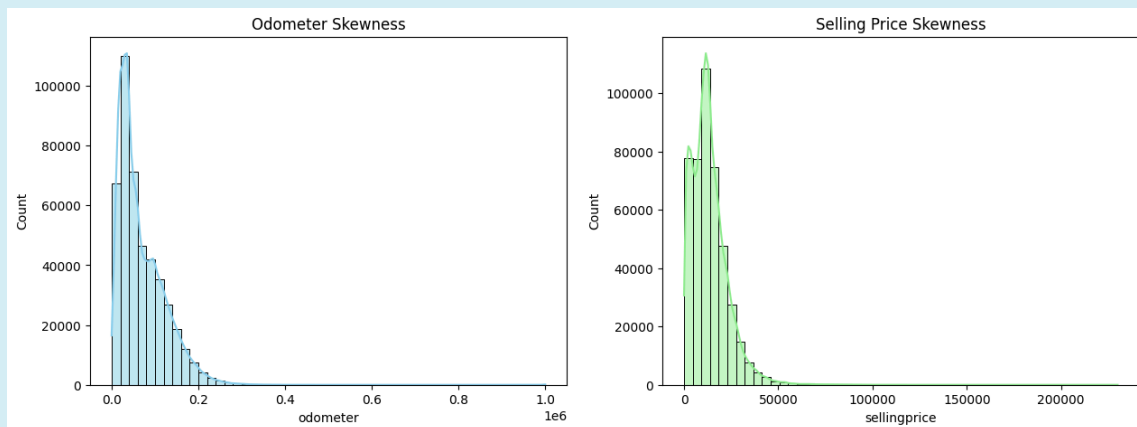
This report outlines the development of the IronLot Auction Pipeline, a data-driven system made to predict the vehicle prices and execute profitable bids in a simulated live auction environment . The core predictive engine is powered by an XGBoost Regressor, selected for its ability to model complex, non-linear depreciation patterns. To ensure robust generalization and prevent geographical overfitting, the model utilizes target encoding and strict data leakage prevention protocols.

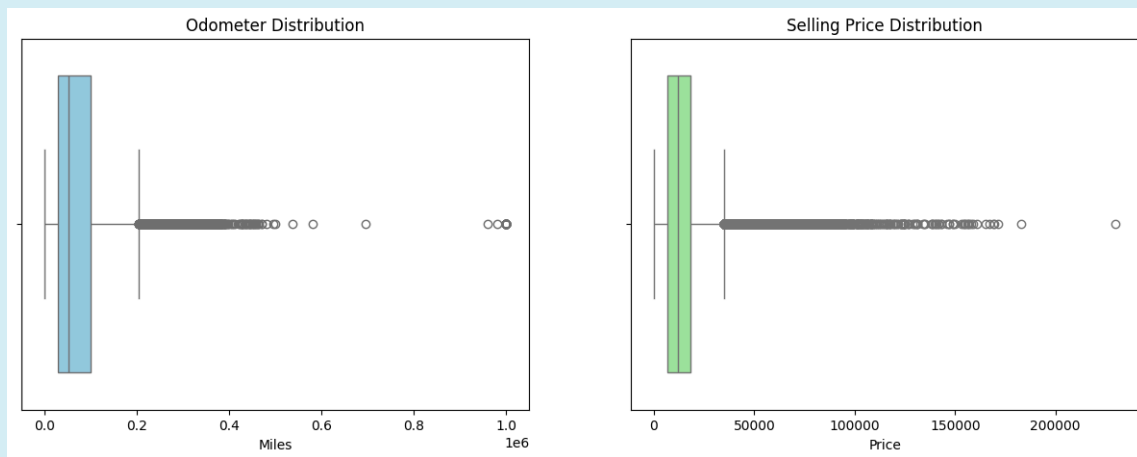
Hyperparameter optimization via RandomisedSearchCV significantly enhances the model's accuracy. The final tuned XGBoost model achieves a Mean Absolute Error(MAE) of \$1,196.33 (an improvement of over \$180 per vehicle compared to the baseline) and reduced the Root Mean Squared Error (RMSE) to \$1,861.53.

The pipeline integrates a bidding agent utilizing an Asymptotic Gap Strategy. This deterministic algorithm mathematically enforces a strict 15% profit margin ceiling while dynamically scaling bid increments based on the auction's round number and remaining budget gap. This approach ensures the agent bids aggressively in early rounds to dominate the auction block, while decelerating logarithmically near the margin ceiling to maximize the profit.

Data Cleaning

Outlier detection & trimming





As demonstrated in the box plots and histograms above, both odometer and sellingprice exhibited severe right-skewness with extreme.

To protect the model from learning from these extreme edge cases, a percentile-based clipping threshold was applied. The top 1% and bottom 1% of the sellingprice (quantiles 0.01 to 0.99) were dropped, along with the top 1% of the odometer distribution. This successfully isolated the true distribution of the data while maintaining a massive sample size for training.

Missing Value Imputation Strategy

	Data Type	Missing Values	Missing %
transmission	str	52299	11.698744
color	str	20361	4.554544
interior	str	14155	3.166327
body	str	10593	2.369544
condition	float64	9437	2.110959
trim	str	8570	1.917020
model	str	8372	1.872730
make	str	8287	1.853716
odometer	float64	69	0.015435

Why imputation?

A requirement of the live auction simulation is that the agent must be able to bid on a vehicle even if the auction feed is missing data fields. Therefore, dropping rows with missing values (dropna()) was strictly prohibited, as it would cause the pipeline to crash in production.

How was it done?

- 1)Missing text fields (such as transmission or body) were imputed with the string 'unknown'.
- 2)Missing continuous variables (odometer and condition) were imputed using the column median. The median was strategically chosen over the mean because, as proven by the histograms, the data remained strictly right-skewed even after outlier clipping. Median is a better statistical metric for skewed distribution (better than mean).

Typo fixing

Unique Body Types	
body	
sedan	193148
suv	114950
hatchback	20952
minivan	20424
coupe	14263
crew cab	13132
wagon	12979
convertible	8351
supercrew	7337
g sedan	5895
supercab	4191
regular cab	3898
extended cab	3597
van	3561
quad cab	3267
e-series van	1465
double cab	1284
g coupe	1269
crewmax cab	450
king cab	417
Name: count, dtype: int64	

Top 15 Car Makes	
make	
ford	75367
chevrolet	48373
nissan	43246
toyota	32047
dodge	24764
honda	21872
hyundai	17429
bmw	16710
kia	14477
chrysler	13999
mercedes-benz	13687
jeep	12418
infiniti	12200
volkswagen	9975
lexus	9564
Name: count, dtype: int64	

Bottom 15 Car Makes	
make	
geo	17
aston martin	17
ferrari	15
rolls-royce	13
gmc truck	10
fisker	8
lamborghini	3
daewoo	2
mercedes-b	2
ford truck	2
airstream	1
dot	1
dodge tk	1
hyundai tk	1
chev truck	1
Name: count, dtype: int64	

An analysis (value_counts) of the categorical columns revealed severe data fragmentation caused by manual entry errors. For example, the make column contained fragmented and misspelled entries such as "mercedes-b", "dot", "hyundai tk", and "chev truck".

Fix: The text was converted into lower case and stripped of the trailing space. Then, using regex, the fix was made like “truck” and “tk”, mathematically aggregated the fragmented rows to their proper parent categories like “dodge tk” to “dodge”, “ford truck” to “ford”, etc.

To reduce the cardinality, using Regex, all variations of truck cabs (e.g., 'crew cab', 'supercrew', 'quad cab', 'king cab') were mapped to a single, unified 'pickup' category. This significantly reduced noise and dimensionality for the model.

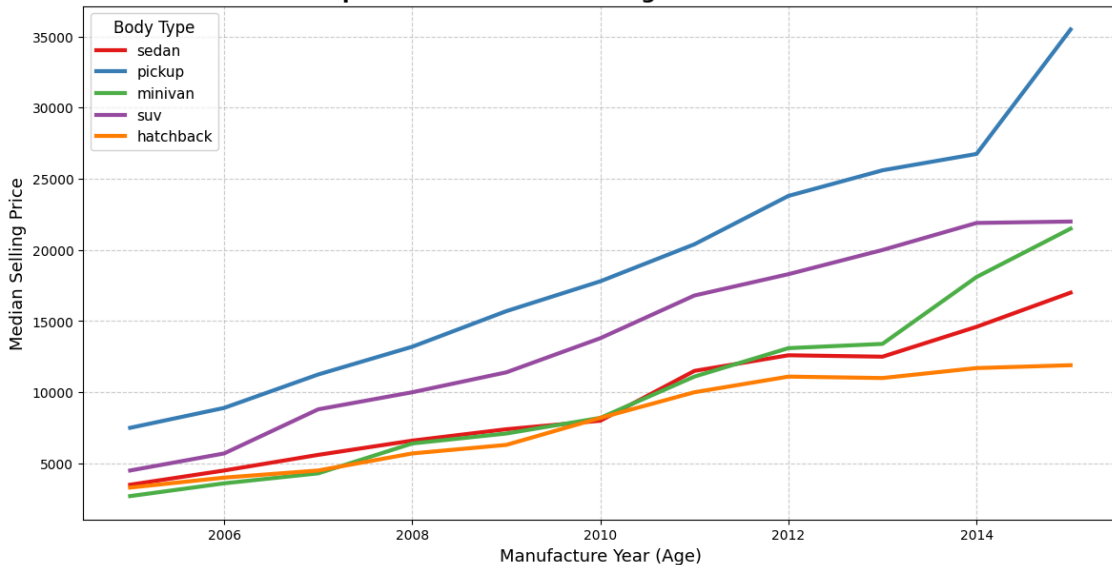
EDA & Feature Engineering

Depreciation dynamics by body type

To build an accurate pricing model, it is critical to understand that time (age) does not affect all vehicles equally.

The median selling price was utilized as the estimator to prevent rare, high-end luxury vehicles from skewing the baseline trend.

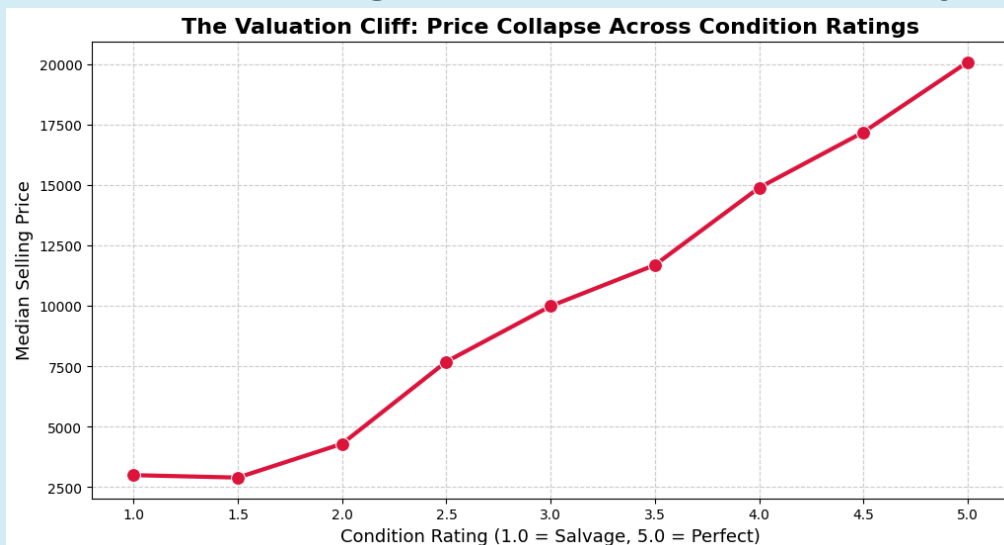
Vehicle Depreciation: Median Selling Price vs. Manufacture Year



For this analysis, the dataset was filtered to include only the top five most common body types manufactured from 2005 onwards. The year cutoff was intentionally applied to remove vehicles older than 20 years, as these vehicles often cross the threshold into "classic car" territory, causing their depreciation curves to invert and their selling value to increase.

This visualization proves that a vehicle's year and its body type have a highly correlated, non-linear relationship. A 10 year old pickup truck retains a vastly different percentage of its original value compared to a 10 year old sedan. Because we are feeding these distinct categories into the XGBoost algorithm via Target Encoding, the model will be perfectly positioned to capture these exact, body-specific depreciation curves when generating final auction bids.

Price across the condition rating (the identification of non-linearity)



To accurately model vehicle valuation, we must analyze how the market prices vary with a vehicle’s physical condition.

The raw condition score is highly granular and continuous decimal values. To resolve the “noise vs signal” problem, the condition rating were converted into a 0.5 resolution scale, restricting the analysis to the valid 1.0 to 5.0 range.

Mathematical proof of Non-Linearity: While the visual inspection of the price curve suggests a relatively steady decline with decline in condition rating, relying solely on visual heuristics is dangerous to identify the non-linearity.

The Rate of Change (0.5 Resolution)			
	sellingprice	dollar_loss	percentage_loss (%)
cond_bucket			
5.00	20100.00	NaN	NaN
4.50	17200.00	2900.00	14.43
4.00	14900.00	2300.00	13.37
3.50	11700.00	3200.00	21.48
3.00	10000.00	1700.00	14.53
2.50	7700.00	2300.00	23.00
2.00	4300.00	3400.00	44.16
1.50	2900.00	1400.00	32.56
1.00	3000.00	100.00	3.45

To rigorously identify non-linear pricing behavior, we calculated the first derivative (the exact percentage rate of change) of the median selling price across the condition buckets.

The tabular output revealed a severe Valuation Cliff at the lower end of the condition spectrum. While standard depreciation occurs steadily between the 5.0 and 3.0 marks. The first derivative calculation proved that a vehicle loses approximately 40% of its total remaining value simply by dropping from a 2.5 to a 2.0 rating.

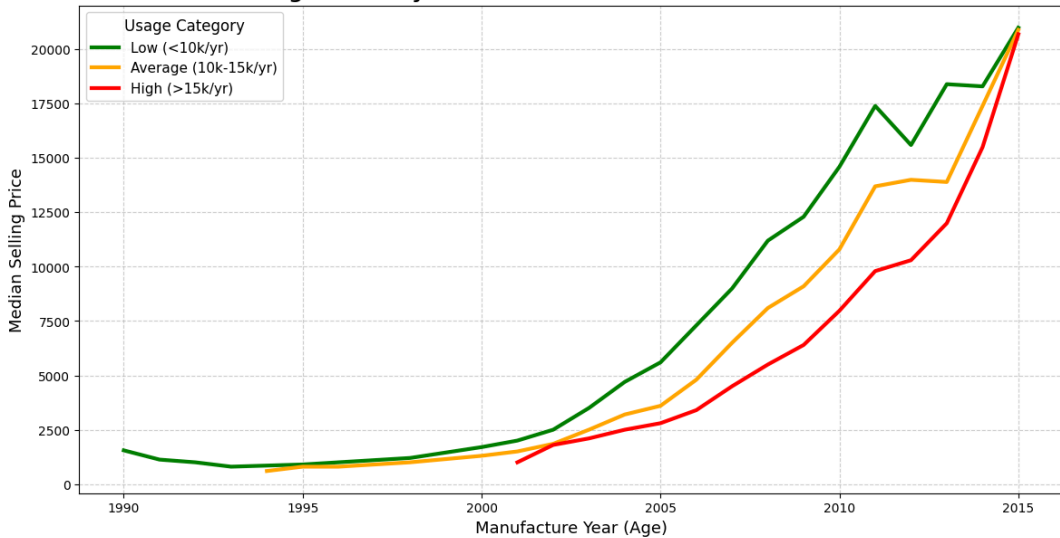
Model rationale idea: This non-linear, cliff-like behavior mathematically validates our choice of a tree-based algorithm (XGBoost). A standard Linear Regression model would attempt to draw a straight line through this curve, mathematically smoothing over the cliff and causing the agent to massively overpay for low-condition vehicles.

Risk Management: This insight acts as a direct warning system for the bidding agent. Vehicles scoring below a 2.5 condition rating exhibit disproportionate financial risk and high price volatility, requiring the agent to strictly defend its 15% profit margin when operating in this sector of the auction block.

Miles per year dictation on selling price

While a raw odometer reading provides a baseline metric of wear, it lacks temporal context. For instance, a vehicle with 60,000 miles could be a heavily abused two-year-old fleet car or a lightly driven ten-year-old commuter. To capture this crucial distinction, we performed active feature engineering to calculate the vehicle's Usage Intensity.

Usage Intensity: How "Miles Per Year" Dictates Valuation



We establish a new `car_age` baseline (using the dataset's maximum year of 2015, with a +1 offset to strictly prevent zero-division errors for brand-new models).

The data was segmented into three distinct usage buckets: Low (<10k/yr), Average (10k-15k/yr), and High (>15k/yr).

As vehicles approach the 15-to-20-year mark, the lines begin to heavily converge. In the older car market, age and condition override the `miles_per_year` metric. Meanwhile, in new cars (about 1 or 1.5 year), the `miles_per_year` does not affect significantly as the new is new, and regardless of how much it is driven, the body and engine are well and good.

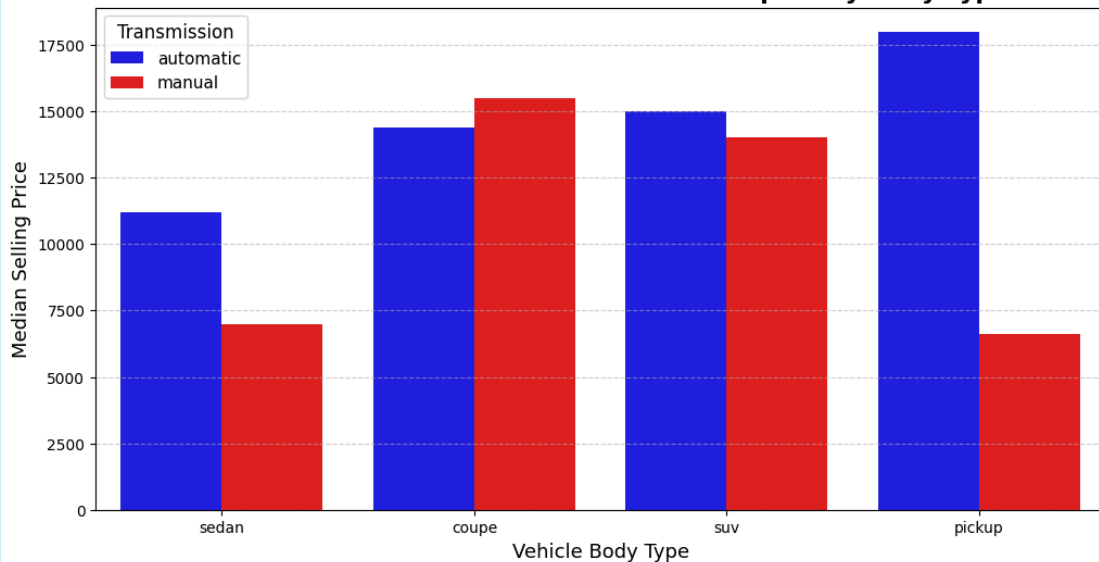
By feeding the XGBoost regressor this engineered velocity metric rather than just a static odometer reading, the algorithm gains a significantly sharper mathematical signal to predict early-life depreciation cliffs.

Transmission vs Body Type

If we evaluate the `transmission` column, a macro-level analysis suggests that automatic vehicles universally have higher auction prices than manuals. However, assuming this rule applies uniformly across the entire market is a statistical trap. To investigate potential anomalies, we mapped the price impact of transmission types strictly across the highest-volume body categories (Sedans, Coupes, SUVs, and Pickups), intentionally filtering out imputed 'unknown' transmissions to ensure a pure A/B comparison.

The anomaly: The Coupe segment completely defies the macro-level market trend. Because Coupes are frequently positioned as sports cars or enthusiast vehicles, a manual transmission is viewed as a desirable feature rather than a drawback.

Niche Market Anomalies: Transmission Impact by Body Type



This visualization serves as the primary mathematical justification for selecting a tree-based algorithm over a standard linear regression model. Linear models struggle with conditional feature interactions. XGBoost, however, builds sequential decision trees capable of handling strict conditional logic.

Data Preprocessing: Encoding & Leakage prevention

Data Leakage prevention: The features matrix (X) and the target variable (y) were separated, and the dataset was partitioned into an 80% training and 20% testing set. It is to note that this separation occurred before any encoding transformations were applied. This strict sequencing guarantees that the model cannot peek at the test data's target values during the encoding process, effectively preventing data leakage.

High-Cardinality Target Encoding: The dataset contains several high cardinality categorical features, such as make, model, and trim. Techniques like One Hot Encoding would have generated thousands of columns, inevitably leading to memory crashes and exposing the model to the dimensionality problem. To solve this, Target Encoding was utilized. Every text category within the training set was replaced by its corresponding median sellingprice.

If the test set or the live simulation encounters a completely new vehicle category that was not present in the training data, the pipeline safely imputes the overall global median price of the training set.

Model Training And Evaluation:

Algorithm Selection & Accuracy Optimization

An XGBoost Regressor was selected over traditional linear models due to its gradient-boosted, tree-based architecture. XGBoost is better for non-linear depreciation patterns (the sharp valuation collapse below a 2.5 condition rating), complex feature interactions (like the manual transmissions retaining value in coupes but dropping in sedans).

To strictly optimize the model's accuracy, a logarithmic transformation (`np.log1p`) was applied to the target variable (`sellingprice`) prior to training. Because auction prices are heavily right-skewed, this transformation normalizes the distribution, stabilizing the loss function and preventing extreme high-end outliers from disproportionately dragging the model's predictions. All final predictions were reverted using an exponential function (`np.expm1`) to calculate real-dollar error metrics.

Geographic Overfitting

Prior to training, the state column was intentionally dropped from the feature matrix. Including geographic data in a pricing model introduces a high risk of geographical overfitting, where the algorithm memorizes local noise rather than true vehicle depreciation metrics. Essentially, geography does influence the `sellingprice` to a high extent but rather it acts more like a noise to the useful data.

Hyperparameter Tuning

Given the massive volume of the dataset (approaching 400,000 rows), deploying an exhaustive `GridSearchCV` was computationally prohibitive. To efficiently explore the hyperparameter space without wasting processing time, a `RandomizedSearchCV` was implemented utilizing a 3-Fold Cross-Validation strategy.

The search space evaluated structural parameters including `n_estimators`, `max_depth`, and `learning_rate`, alongside regularization parameters like `subsample` and `colsample_bytree` to prevent the trees from memorizing the training data.

Best Hyperparameters Found: {'subsample': 1.0, 'n_estimators': 500, 'max_depth': 7, 'learning_rate': 0.1, 'colsample_bytree': 0.8}

Performance Evaluation

The tuning pipeline successfully optimized the algorithm, yielding a significant reduction in error compared to the default XGBoost parameters.

DEFAULT MODEL (Log-Transformed):

MAE: \$1,427.33

RMSE: \$2,180.15

TUNED MODEL (Log-Transformed):

MAE: \$1,227.35

RMSE: \$1,893.74

Bid Sequence Design (The Agent) & Mathematics

The pipeline required translating the XGBoost price predictions into a live, automated bidding strategy. The LiveAuctionAgent was engineered to operate for this specific purpose.

In a live auction simulation, executing complex pandas operations (like `.groupby()`) during runtime introduces unacceptable latency. To achieve fast runtime, the agent strictly separates the training environment from the production environment. When a vehicle enters the auction block, the simulation provides a singular data dictionary rather than a structured DataFrame. The agent maps these incoming features strictly to the format expected by the XGBoost model. A strict mathematical fallback mechanism was implemented for target encoding. If the live feed presents a categorical string previously unseen in the training data the agent bypasses KeyErrors by appending the global median value for that specific category. The model's raw output (log dollars) is then immediately converted back to a real-dollar prediction using `np.exp(m1)`.

The Asymptotic Gap Strategy

We used Asymptotic Gap Strategy instead of Flat Increment bidding or Max Bid Bidding strategy. The Asymptotic Gap Strategy was designed to perfectly balance Aggression (winning the car) with Margin Defense (protecting your 15% profit). It solves the problems above by changing the bid size dynamically based on the state of the auction. In Asymptotic Gap Strategy, when the auction starts, the gap between the current price and your ceiling is huge. The strategy takes a large percentage of that gap and bids it immediately. As the current price gets closer and closer to your \$8,500 ceiling, the "Gap" shrinks. Because your bid increment is a percentage of that gap, your bids automatically get smaller and smaller.

The Bidding formula

To optimize win rates while fiercely defending the portfolio bankroll, the agent utilizes a custom deterministic fractional gap sequence.

- ☆ The margin ceiling (M): A strict 15% minimum profit margin is mathematically enforced.

$$M = \min(0.85 \times V_{\text{pred}}, B),$$

V_{pred} is XGBoost predicted value
 B is available bankroll.

- ☆ The gap (G): The agent calculates the remaining bidding room:

$$G = M - C, \quad C \text{ is the current highest bid}$$

If $G \leq 0$, the agent folds.

- ☆ Dynamic Round weighting (W):

$$W = \min(0.5, 0.1 \times R), \quad R \text{ is the internal auction round tracker}$$

- ☆ The Final Bid:

$$\text{NextBid} = C + (G \times W)$$

Theoretical advantages over Flat Increments

Traditional flat-increment bidding (ex. always bidding exactly \$100 more) is mathematically inefficient. It wastes processing time through high-frequency micro-bidding and actively invites prolonged bidding wars with competing agents. The Asymptotic Gap Sequence mathematically solves this by reacting dynamically to the remaining spread rather than relying on a static integer.

Aggression vs. Margin Trade-Offs

This formula balances the fundamental trade-off between securing the win (aggression) and protecting ROI (margin).

Early Round Aggression: By multiplying the Gap by the Round Weight, the agent takes massive, aggressive leaps in early rounds (when the gap is huge) to instantly crush weak opponents and dominate the auction block.

Late Round Margin Defense: As the current bid approaches the Margin Ceiling, the gap shrinks to near zero. Even at maximum weight, multiplying a tiny gap results in a tiny increment. The agent automatically decelerates into precise micro bids, ensuring the 15% profit margin is never accidentally breached.

